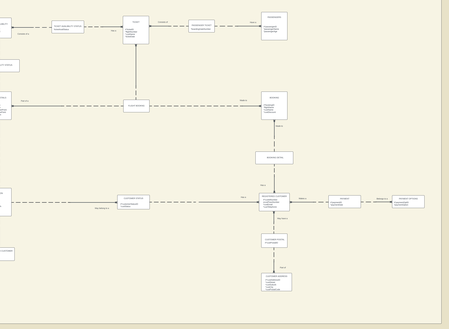
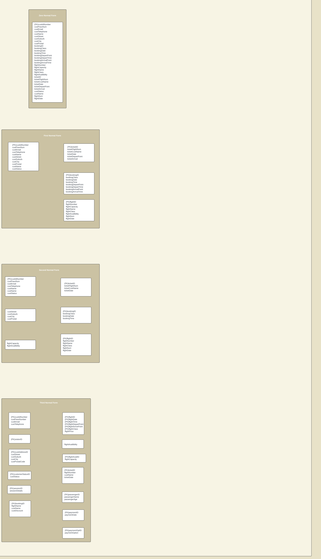
Muhammed Uwais Adam
Project 3: Airline Booking System
eduv136908

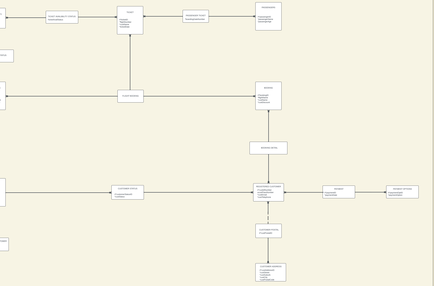
Initial ER diagram



Normalisation



ER diagram from 3NF



Mapping to tables

AIRPORTS_CUSTOMER

Column Name	Value
Row Type	FK
Key Type	FK
Table	AIR
Foreign Key	4
Nullable	N

BOOKING

Column Name	AirportID	AirportName	AirportCity	AirportCountry	IsDST	IsDeleted	IsDST1	IsDeleted1	FlightDurationID	FlightDurationID1	IsDeleted2
Row Type	PK	NN	NN	NN	NN	NN	NN	NN	NN	NN	NN
Table	AIR					Fly	AIR	FB	Fly	AIR	FB
Foreign Key	Fly.Departure Fly.Arrival					AIR.AirportID	1.0	2.0000	2.0.0	AIR.FlightID	2.0000
Nullable	N	N	N	N	N	N	Unique	2.0.0	Unique	Fly.FlightID	Unique

FLIGHT DETAILS

Column Name	IsDST	IsDeleted	IsDST1	IsDeleted1	IsDST2	IsDeleted2
Row Type	NN	NN	NN	NN	NN	NN
Table	FB		FB		FB	
Foreign Key	2.0000	2.0.0	AIR.AirportID	1.0	AIR.AirportID	2.0
Nullable	Unique	2.0.0	Unique	1.0	Unique	2.0

FLIGHT DURATION STATUS

Column Name	IsDST	IsDeleted	IsDST1	IsDeleted1	IsDST2	IsDeleted2
Row Type	NN	NN	NN	NN	NN	NN
Table	AIR		AIR		AIR	
Foreign Key	AIR.AirportID	1.0	AIR.AirportID	2.0	AIR.AirportID	3.0
Nullable	Unique	1.0	Unique	2.0	Unique	3.0

FLIGHT MANUALLY

Column Name	IsDST	IsDeleted
Row Type	NN	NN
Table	FB	
Foreign Key	2.0	2.0
Nullable	2.0	2.0

FLIGHT DURATION STATUS

Column Name	IsDST	IsDeleted	IsDST1
Row Type	NN		NN
Table	FB		FB
Foreign Key	2.0	FlightID	2.0
Nullable	2.0	No Nullable	2.0

FLIGHT

Column Name	IsDST	IsDeleted	IsDST1	IsDST2
Row Type	NN		NN	
Table	FB		FB	FB
Foreign Key	AIR	2.0	AIR	AIRPORTS
Nullable	AIR	2.0	AIR	AIRPORTS

PASSENGER PAYMENT

Column Name	IsDST	IsDeleted	IsDeleteDeleted
Row Type	NN		NN
Table	FB		FB
Foreign Key	AIR	2.0	NN
Nullable	AIR	2.0	NN

PASSENGER

Column Name	IsDeleted	IsDeleted1	IsDeleted2
Row Type	2.0		
Table	FB	FB	FB
Foreign Key	2.0	NN	NN
Nullable	2.0	NN	NN

FLIGHT BOOKING

Column Name	IsDST	IsDeleted	IsDST1	IsDeleteDeleted	IsDeleteDeleted1	IsDeleted	IsDeleted1	IsDeleted2
Row Type	NN		NN	NN	NN	NN	NN	NN
Table	FB		FB	FB	FB	FB	FB	FB
Foreign Key	AIR	2.0	AIR	AIR	AIR	Bookings	Bookings	2.0
Nullable	AIR	2.0	AIR	AIR	AIR	Bookings	Bookings	2.0

BOOKING

Column Name	IsDeleted	IsDeleted1	IsDeleted2	IsDeleted3
Row Type	NN			
Table	FB	FB	FB	FB
Foreign Key	2.0	NN	NN	NN
Nullable	2.0	NN	NN	NN

BOOKING DETAIL

Column Name	IsDeleted
Row Type	NN
Table	FB
Foreign Key	2.0
Nullable	2.0

BOOKING STATUS

Column Name	LastUpdated	LastUpdated1	LastUpdated2	LastUpdated3
Row Type	NN			
Table	FB		FB	FB
Foreign Key	BOOKINGS		BOOKINGS.BookingID	BOOKINGS
Nullable	BOOKINGS		BOOKINGS.BookingID	BOOKINGS

BOOKS

Column Name	LastUpdated	LastUpdated1	LastUpdated2	LastUpdated3
Row Type	NN		NN	NN
Table	FB	FB	FB	FB
Foreign Key	2.0	BOOKINGS	BOOKINGS	2.0
Nullable	2.0	BOOKINGS	BOOKINGS	2.0

BOOKS STATUS

Column Name	LastUpdated	LastUpdated1
Row Type	NN	
Table	FB	FB
Foreign Key	2.0	NN
Nullable	2.0	NN

CATEGORIES

Column Name	LastUpdated	LastUpdated1	LastUpdated2
Row Type	NN	NN	NN
Table	FB	FB	FB
Foreign Key	BOOKINGS	BOOKINGS	2.0
Nullable	BOOKINGS	BOOKINGS	2.0

CUSTOMER SERVICE

Column Name	LastUpdated	LastUpdated1	LastUpdated2	LastUpdated3
Row Type	NN			
Table	FB	FB	FB	FB
Foreign Key	2.0	BOOKINGS	BOOKINGS	NN
Nullable	2.0	BOOKINGS	BOOKINGS	NN

Documentation

Author: Muhammed Uwais Adam
Date: 2025-08-07
Project: Airline Booking System

The purpose of this project is to design a relational database system for a South African Flight Booking Agency.

The Agency's public Customers or the Agent based in cities across the country planned visit some provinces.

Customers can book flights online or through a dedicated customer service office.

Every flight can accommodate several crew.

The crew is mostly made up of the information about flight details, the ID of every flight Passenger is created by customer (case registered) and compensated.

The passenger enters data of information for the flight details the customer (agent) enters. Compensation is paid to crewmen for the flight, depending on the distance covered and the duration of every flight.

Among a distinction the agency has also which is operated. Compensation is paid.

Every flight has a unique identifier, departure and arrival airports, departure and arrival times, and the status (active, delayed, cancelled).

Airports are identified by their IATA codes (three-letter codes) and ICAO codes (four-letter codes), and they include details such as the city, country, and geographical coordinates (latitude and longitude).

Flight durations are calculated based on the departure and arrival times, and the system may also track the actual flight duration for analysis purposes.

Passengers can book multiple flights in a single booking, and each booking can have different passenger details and payment methods. The system should track the booking status (confirmed, cancelled, pending).

The Passenger entity stores personal information, including their name, date of birth, gender, nationality, and passport details. Contact information (email and phone number) is also stored for communication purposes.

The Payment entity records all payment transactions, including the payment amount, date, method (credit card, debit card, bank transfer, etc.), and status (successful, pending, failed). The system should support multiple payment methods for a single booking.

The Flight Booking entity links the passenger, flight, and payment details together. The system should ensure that the booking date is not in the future.

The Customer Service Representatives table stores information about the agents who handle customer inquiries and bookings. The system should track their hire date and status (active, inactive).

The Customer Service Types table defines different types of customer service interactions (e.g., booking assistance, complaint resolution, general inquiry).

The Customer Service Status table defines the possible statuses for customer service interactions (e.g., open, in progress, resolved, closed).

The Customer Service Interactions table records all customer service interactions, including the type of interaction, the representative involved, the date and time, and the status.

Indexes should be created on frequently queried columns, such as flight_date, departure_airport, arrival_airport, passenger_email, and booking_status, to improve query performance.

Foreign key constraints must be enforced to maintain referential integrity between related entities.

Check constraints can be used to enforce business rules, such as ensuring that the departure time is earlier than the arrival time, and that the passenger age is within a reasonable range.

Consider implementing audit tables to track changes to critical data, such as flight status updates and booking modifications.

Implement appropriate data types for each attribute, ensuring that dates are stored as DATE or TIMESTAMP, monetary values are stored as DECIMAL or NUMERIC, and text fields have appropriate VARCHAR lengths.

Consider implementing soft deletes (using a deleted_at timestamp) rather than hard deletes for important entities, to maintain historical data.

Implement a backup and recovery strategy to ensure data can be restored in the event of a system failure.

Consider implementing a data warehouse for analytical queries, separate from the transactional database, to avoid impacting performance.

Implement appropriate indexing strategies for the most frequently accessed tables.

Consider implementing a caching layer for frequently accessed data, such as airport information and flight schedules.

Implement proper error handling and logging mechanisms.

Consider implementing a microservices architecture for different functions (booking, payment, customer service) for better scalability.

Implement proper authentication and authorization mechanisms.

Consider implementing a data lake for storing historical data and performing analytics.

Implement proper data encryption for sensitive information (passport numbers, payment details).

Consider implementing a recommendation system based on passenger preferences and booking patterns.

Implement proper monitoring and alerting for system performance and errors.

Consider implementing a mobile-friendly interface for customers.

Implement proper documentation for all tables, columns, and relationships.

Consider implementing a data quality framework to ensure data integrity.

Implement proper testing (unit tests, integration tests, and end-to-end tests).

Consider implementing a CI/CD pipeline for automated deployments.

Implement proper version control for database schema changes.

Consider implementing a data governance framework.

Implement proper access control (role-based access control).

Consider implementing a disaster recovery plan.

Implement proper documentation for the system architecture.

Consider implementing a data catalog to help users find and understand data.

Implement proper data lineage tracking.

Consider implementing a machine learning model for predicting flight delays.

Implement proper data quality checks.

Consider implementing a real-time analytics dashboard.

Implement proper error handling and retry mechanisms.

Consider implementing a data mesh architecture.

Implement proper data masking for sensitive information in non-production environments.

Consider implementing a feature store for machine learning models.

Implement proper monitoring of database performance.

Consider implementing a data warehouse for historical analysis.

Implement proper documentation for the ETL processes.

Consider implementing a data lakehouse architecture.

Implement proper data retention policies.

Consider implementing a data catalog for better data discovery.

Implement proper documentation for the data models.

Consider implementing a data quality framework.

Implement proper testing for the ETL processes.

Consider implementing a data governance framework.

Implement proper access control for the data warehouse.

Consider implementing a disaster recovery plan for the data warehouse.

Implement proper documentation for the data warehouse.

Consider implementing a data catalog for the data warehouse.

Implement proper data lineage tracking for the data warehouse.

Consider implementing a machine learning model for predicting customer churn.

Implement proper data quality checks for the data warehouse.

Consider implementing a real-time analytics dashboard for the data warehouse.

Implement proper error handling and retry mechanisms for the data warehouse.

Consider implementing a data mesh architecture for the data warehouse.

Implement proper data masking for sensitive information in the data warehouse.

Consider implementing a feature store for machine learning models in the data warehouse.

Implement proper monitoring of database performance in the data warehouse.

Consider implementing a data warehouse for historical analysis in the data warehouse.

Implement proper documentation for the ETL processes in the data warehouse.

Consider implementing a data lakehouse architecture in the data warehouse.

Implement proper data retention policies in the data warehouse.

Consider implementing a data catalog for better data discovery in the data warehouse.

Implement proper documentation for the data models in the data warehouse.

Consider implementing a data quality framework in the data warehouse.

Implement proper testing for the ETL processes in the data warehouse.

Consider implementing a data governance framework in the data warehouse